Clarify-or-Code: An Issue-to-PR Agent That Knows When to Ask

Project overview, tech stack, phased plan and milestones

1. What you are building

A multi-agent system (a LangGraph supervisor, specialist agents and MCP tool servers) that takes a GitHub issue, finds the relevant code in a repository, writes a patch, runs the repository's tests inside a locked-down Docker container, repairs failures within a retry limit, and opens a **draft** pull request on a repository you own. It never merges anything.

The distinguishing idea is a **clarification gate**: before writing code, the agent decides whether the issue is clear enough to proceed or whether it should ask the developer one question. You then measure whether asking improves the resolve rate.

Initial scope (keep it narrow): Python repositories with working pytest suites, clear bug-fix issues, public repos forked to your own account, hosted LLM (no local model, no GPU needed).

2. Tech stack

Component	Technology	Purpose
Language and environment	Python 3.12, uv	Core code, locked dependencies
Orchestration	LangGraph with SQLite checkpointer	Stateful graph, conditional edges, retries, `interrupt()` pause and resume
LLM access	LiteLLM with one strong hosted code model and one cheap model	Patching, issue analysis, review; set a hard cost cap per run
Repository operations	Git CLI via subprocess	Clone, branch, diff, commit, push
GitHub	PyGithub (REST API)	Fetch issues, create draft PRs; PyDriller to mine benchmark tasks

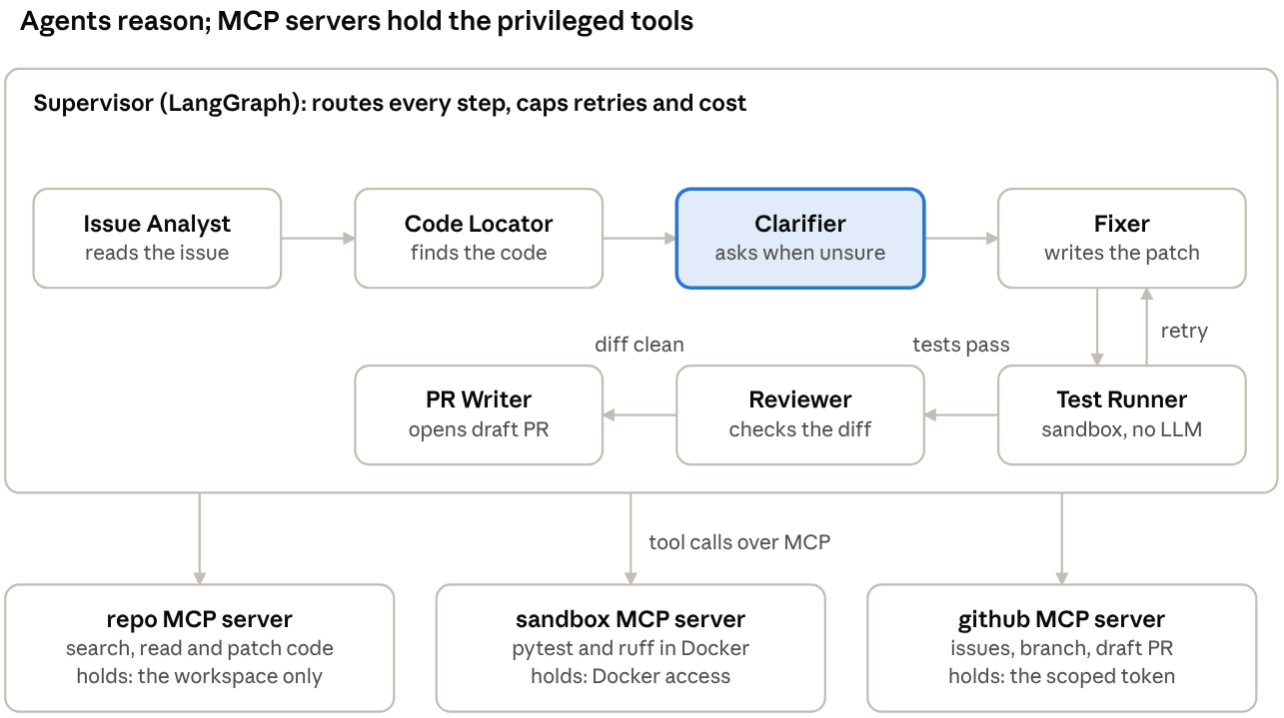
Component	Technology	Purpose
Code parsing	Python `ast`, then tree-sitter	Function and class level chunks
Retrieval	rank_bm25, then sentence-transformers and FAISS	Locate relevant code; hybrid later
Sandbox	Docker Engine and the Docker Python SDK	Isolated test execution
Quality checks	pytest, ruff	Validate patches and the project itself
Clarification gate	scikit-learn, LightGBM, SHAP, pandas	Predict when a first attempt is likely to fail
Service and UI	FastAPI, Streamlit	Run the workflow, show the graph trace
Logging and results	Structured JSON logs, Logfire optional, SQLite or Parquet	Traces, cost, benchmark results
Voice (final phase)	Gnani Prisma streaming STT, Timbre TTS, rapidfuzz	Spoken clarification; snap transcribed names to real code symbols
Tool layer (MCP servers)	Official mcp Python SDK (FastMCP)	Repo, sandbox and GitHub tools behind permission boundaries
MCP client	langchain-mcp-adapters (MultiServerMCPClient)	Loads MCP tools into LangGraph agents with per-agent allowlists

Needed for the MVP: Python, LangGraph, an LLM provider, Git, GitHub API, Docker, pytest, SQLite, and the MCP SDK. Everything else can wait.

3. Multi-agent architecture

The system is a LangGraph supervisor graph: one Supervisor routes work to specialist agents, each with its own prompt, model tier, tool allowlist and typed output. Agents hand work to each other through shared graph state, not free-form chat.

Agent	Job	Tools (through MCP)	Model
Supervisor	Routes between agents, enforces retry and cost limits, decides when to stop	None; reads graph state only	Cheap
Issue Analyst	Extracts the expected behaviour from the issue and comments, scores how ambiguous it is	GitHub read tools	Cheap
Code Locator	Finds the files and functions most likely involved	Repo search and read tools	Cheap
Clarifier	Asks the developer one question when the gate says the risk is high (voice in Phase 9)	`interrupt()` in LangGraph	Cheap
Fixer	Writes the plan and patch, applies edits, repairs from test failures	Repo read and patch tools	Strong
Test Runner	Runs pytest and ruff in the sandbox and returns logs; plain code, no LLM	Sandbox tools	None
Reviewer	Checks the diff for unrelated edits, test tampering and regressions	Repo read-only tools, ruff	A different model from the Fixer
PR Writer	Drafts the PR description and opens the draft PR	GitHub write tools, draft PR only	Cheap



agent flow · supervisor, 7 workers, 3 MCP servers

The highlighted Clarifier is the step this project is built around. No agent holds a token or Docker access; every tool call goes through an MCP server.

Design rules

- Agents are LangGraph nodes or subgraphs sharing typed state, not separate processes. Each returns a schema-validated object before the next agent runs.
- Each agent sees only the tools and context its job needs.
- The Reviewer runs on a different model from the Fixer, so one model's blind spots are less likely to pass its own work.
- The Test Runner is plain code, not an LLM. Calling it an agent would overstate it.
- Every extra agent adds LLM calls and latency, so log cost per resolved issue from the start.
- Multi-agent is not automatically better. Phase 8 runs a single-agent baseline against this design, so you can show what the split bought, or report honestly that it did not help.

4. MCP tool layer

Tools live in three MCP servers, and the agents call them through an MCP client. This keeps privileged operations out of the agents' reach and makes the tools reusable from Claude Desktop or an IDE.

MCP server	Tools exposed	Holds
repo	search_code, read_file, list_symbols, apply_patch, git_diff	The cloned workspace; no network, no token
sandbox	run_tests, run_lint, get_logs	Docker access; the only component that starts containers
github	get_issue, list_comments, create_branch, push_branch, open_draft_pr	The scoped GitHub token, limited to your fork

Rules:

- Per-agent tool allowlists: the Fixer can patch but not call GitHub, the Reviewer is read-only, and only the PR Writer can open a draft PR.
- Tool outputs (issue text, file contents, logs) are untrusted data, never instructions to an agent.
- Log every tool call with its caller, arguments and result.
- Build the servers with the official `mcp` Python SDK (FastMCP) and load their tools into LangGraph with `langchain-mcp-adapters`. Use stdio locally and the HTTP transport if you deploy.
- MCP is plumbing, not intelligence. Plain function tools would be simpler. The reasons to use MCP here are the permission boundary, tool reuse and a protocol interviewers ask about.

5. Sandbox rules (non-negotiable)

- Build the environment image with dependencies installed first, then run tests with **no network**.
- Run as a non-root user with CPU, memory and process limits, a timeout and a read-only root filesystem.
- Never mount the Docker socket or the host filesystem broadly, and never pass your GitHub token into the container.
- Mount test files **read-only** so the agent cannot edit tests to make them pass.
- Treat issue text and comments as untrusted input. Scope the GitHub token to a single test repository.

6. Phases

Phase 0: Foundation

Create the repository, set up uv, define the LangGraph state schema (issue, repo path, relevant files, patch, test results, retry count, review result, PR URL, status), fix the scope above, and write down the agent roster and each agent's output schema.

Phase 1: Issue ingestion and repository setup

Accept an issue URL, parse owner, repo and number, fetch title, body and comments, clone the repo, and create a working branch. Expose the GitHub calls as tools of the github MCP server from the start.

Phase 2: Code localization

Scan the repo, exclude junk directories, extract functions and classes with `ast`, and rank relevant code with BM25. Produce a bounded context package for the LLM. Expose search and file reading through the repo MCP server. The Issue Analyst and Code Locator are separate agents from here.

Phase 3: Patch generation

Have the LLM write a short plan, then apply real edits to the working copy using edit tools. Inspect the actual git diff afterwards. The Fixer agent edits only through the repo server's patch tool.

Phase 4: Docker testing and repair loop

Run tests in the sandbox, capture output, classify results, and feed failures back for up to three repair attempts. Treat infrastructure failures (broken image, missing dependency) separately from code failures. Run tests only through the sandbox MCP server, so the Fixer never touches Docker.

Phase 5: Draft PR and baseline run

Run final validation, add the Reviewer agent (it checks the diff for unrelated edits and test tampering), have the PR Writer agent write a clear PR description (issue link, files changed, tests run, limitations), push the branch to your fork and open a draft PR. Run the agent on 10 to 15 resolved issues as a first sanity check.

> **The initial working version is ready at the end of Phase 5.** At that point you can give it an issue URL and get a real draft PR, within the supported scope. Do not add features until you have tested it on real tasks. The MVP already includes the agent roster and the three MCP servers.

Phase 6: Benchmark and evaluation

Build 40 to 60 tasks from 5 to 8 small Python repos: check out the commit before each fix, use the original issue text, hide the fix and its tests. Prefer lesser-known repos to reduce the chance the model memorised the solution. Track resolve rate, first-attempt success, repair success, cost per issue, retries and wall time. Run several seeds and report confidence intervals and sample size honestly. Start building the task set early, alongside Phase 5.

Phase 7: Better retrieval

Add tree-sitter chunking, embeddings with FAISS, hybrid BM25 plus vector ranking and reranking. Measure Recall@5 and Recall@10 on whether the truly modified files appear in results, and compare against BM25 alone.

Phase 8: Clarification gate and ablation

Add an `interrupt()` step that can ask the developer one question. Compare four arms on every task: never ask, always ask, rule-based ask and learned gate (LightGBM on features such as retrieval score gap, issue length and presence of a stack trace, with SHAP to explain it). For evaluation, a simulated developer (an LLM that sees the true fix) answers questions; state this limitation. Also compare with and without the retry loop, and the single-agent baseline against this multi-agent design, with and without the Reviewer.

> **The project is resume-ready at the end of Phase 8.** You now have a working system, a results table and an ablation.

Phase 9: Voice layer (about two days)

Let the developer answer the clarifying question by speaking in Hinglish through Prisma, hear the question through Timbre, and snap transcribed identifiers to real repo symbols with rapidfuzz. Check Gnani's documentation for custom vocabulary support before relying on it.

Phase 10: Optional extras (only if time remains)

A polished Streamlit or FastAPI interface, an outer MCP server that lets Claude Desktop or an IDE submit an issue to the whole system, and deeper review with static analysis and coverage. Skip webhooks, authentication and multi-language support; they add effort without strengthening the story.

7. Rough timeline

Milestone	Phases	Approximate time
Working end-to-end MVP	0 to 5	10 to 12 days
Measured and resume-ready	6 to 8	8 to 11 days
Voice demo and write-up	9	2 days

These are estimates for focused work; environment setup per repo is usually where time slips.

8. What to say in an interview

Lead with the question you tested: does asking a clarifying question improve resolve rate, and can a learned gate beat always asking? Show the results table, then the failure analysis (patches that pass tests but are wrong, setup failures, retry loops, test tampering), then the sandbox security decisions. Be ready to explain why a graph with a checkpointer is better than a simple loop, which parts you built yourself, what the multi-agent split and MCP gave you (permission boundaries, independent review, reusable tools), and the ablation that shows whether the split helped.

Only put numbers on your CV that you actually measured.